

Mlvan Rojas's Org
Scan Report



Application: monitoreoycontrolidc2026.registraduria.gov.co

Environment: Pre-Production

Completed On: Wed, 30 Jul 2025 15:09:11 GMT

Application: monitoreoycontrolidc2026.registraduria.gov.co

Environment: Pre-Production

Scan Date: Wed, 30 Jul 2025 15:09:11 GMT | Paths Scanned: 20 | Scan Duration: 2038s

High	Medium	Low	New	Assigned	Risk Accepted	False Positive
1	8	14	23	0	0	0

Criticality	CWE	Finding	New	Assigned	Risk Accepted	False Positive
High	CWE-89	SQL Injection - Oracle - Time Based	1	0	0	0
Medium	CWE-693	Content Security Policy (CSP) Header Not Set	8	0	0	0
Low	CWE-614	Cookie Without Secure Flag	2	0	0	0
Low	CWE-1275	Cookie Without SameSite Attribute	2	0	0	0
Low	CWE-1004	Cookie No HttpOnly Flag	2	0	0	0
Low	CWE-693	Permissions Policy Header Not Set	8	0	0	0

SQL Injection - Oracle - Time Based

Criticality: High CWE ID: 89

Description:

Remediation

- 1. Use built-in Object Data Models:** Instead of directly passing user input to the back-end SQL server, use built-in Object Data Models to gather and handle data. This helps to ensure that the input is properly sanitized and validated before being used in SQL queries.
- 2. Parameterized queries:** Use parameterized queries in the language framework to construct SQL statements. This helps to prevent SQL injection by separating the SQL code from the user input. Here's an example in Python using the cx_Oracle library:

```
import cx_Oracle
# Establish a connection to the Oracle database
connection = cx_Oracle.connect("username", "password",
"hostname:port/service_name")
# Create a cursor object
cursor = connection.cursor()
# Use a parameterized query to retrieve data
query = "SELECT * FROM users WHERE username = :username"
cursor.execute(query, username="admin")
# Fetch the results
results = cursor.fetchall()
# Close the cursor and connection
cursor.close()
connection.close()
```

- 3. Avoid string concatenation:** Avoid directly concatenating user input with SQL statements in the code base. This can make the application vulnerable to SQL injection attacks. Instead, use parameterized queries or stored procedures to handle dynamic SQL statements.

About

Most SQL injection vulnerabilities occur when input from the user is not properly sanitized and is directly passed to the back-end SQL server. This allows attackers to manipulate the input and inject malicious SQL commands into the application.

Risks

SQL injection vulnerabilities can lead to various risks, including:

- Enumeration of column names:** An attacker can use SQL injection to gather information about the database structure, such as the names of tables and columns. This information can be used for further attacks or unauthorized access.
- Data retrieval:** By injecting malicious SQL commands, an attacker can retrieve sensitive data from the database, such as usernames, passwords, or other confidential information.
- Remote code execution:** In some cases, SQL injection vulnerabilities can be exploited to execute arbitrary code on the server, leading to complete compromise of the application and potentially the underlying system.

Most StackHawk tests for SQL injection vulnerabilities are time-based. This means that the scanner tries to make the SQL server perform time-based actions and observes the response time of the application to detect potential SQL injection vulnerabilities.

Path	Method	Status
/jakarta.faces.resource/primeicons/primeicons.css?ln=primefaces / (SELECT UTL_INADDR.get_host_name('10.0.0.1') from dual union SELECT UTL_INADDR.get_host_name('10.0.0.2') from dual union SELECT UTL_INADDR.get_host_name('10.0.0.3') from dual union SELECT UTL_INADDR.get_host_name('10.0.0.4') from dual union SELECT UTL_INADDR.get_host_name('10.0.0.5') from dual) &v=13.0.6	GET	NEW

Content Security Policy (CSP) Header Not Set

Criticality: Medium CWE ID: 693

Description:

Remediation

To fix the "Content Security Policy (CSP) Header Not Set" vulnerability, you need to properly configure the Content Security Policy header in your web server or application. Here are the steps to remediate this issue:

1. Identify the web server or application: Determine the technology stack used for your website or application. This could be Apache, Nginx, IIS, or a specific web framework like Express.js or Django.

2. Configure the Content Security Policy header: Add the Content Security Policy header to your server or application configuration. The exact method will depend on the technology stack you are using.

- For Apache, add the following line to your .htaccess file or Apache configuration file:

```
Header set Content-Security-Policy "default-src 'self';"
```

- For Nginx, add the following line to your server block or location block:

```
add_header Content-Security-Policy "default-src 'self';";
```

- For IIS, open the IIS Manager, select your website, and go to the "HTTP Response Headers" section. Add a new header with the name "Content-Security-Policy" and the value "default-src 'self';".

- For specific web frameworks, consult the documentation for that framework on how to set the Content Security Policy header.

3. Customize the Content Security Policy: The example above sets a basic Content Security Policy that only allows resources to be loaded from the same origin. However, you may need to customize the policy based on your website's requirements. Refer to the Content Security Policy specification (<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>) for more information on the available directives and their usage.

4. Test and monitor: After configuring the Content Security Policy header, thoroughly test your website or application to ensure that all resources are being loaded correctly. Monitor your application's security logs and reports to detect any potential violations or issues with the Content Security Policy.

About

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks. These attacks can be used for various malicious purposes, such as data theft, site defacement, or distribution of malware.

CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on a page. The covered types of content include JavaScript, CSS, HTML frames, fonts, images, and embeddable objects like Java applets, ActiveX, audio, and video files.

By properly configuring the Content Security Policy header, you can restrict the loading of content from unauthorized sources and reduce the risk of XSS attacks and other malicious activities.

Risks

Failure to set the Content Security Policy (CSP) header exposes your website or application to various security risks:

- 1. Cross-Site Scripting (XSS) attacks:** Without a proper CSP, attackers can inject malicious scripts into your web pages, leading to XSS attacks. These attacks can result in the theft of sensitive user information, session hijacking, or defacement of your website.
- 2. Data injection attacks:** Attackers can inject malicious data into your website or application, leading to data corruption, unauthorized access, or the execution of arbitrary code.
- 3. Malware distribution:** Without a CSP, attackers can load and execute malicious files on your website, leading to the distribution of malware to your users.
- 4. Site defacement:** Attackers can modify the appearance or content of your website, damaging your brand reputation and causing user distrust.

By implementing a proper Content Security Policy, you can significantly reduce the risk of these security threats and protect your website or application from potential attacks.

Path	Method	Status
[ROOT]	GET	NEW
/index	GET	NEW
/invalidsession	GET	NEW
/jakarta.faces.resource/css/bootstrap.min.css	GET	NEW
/jakarta.faces.resource/css/oneui.css	GET	NEW
/jakarta.faces.resource/theme.css?ln=primefaces-saga&v=13.0.6	GET	NEW
/robots.txt	GET	NEW
/sitemap.xml	GET	NEW

Cookie Without Secure Flag

Criticality: Low CWE ID: 614

Description:

Remediation

To remediate the vulnerability "Cookie Without Secure Flag," the following steps can be taken:

- 1. Enable the secure flag for cookies:** Ensure that the secure flag is set for all cookies that contain sensitive information or require secure transmission. This can be done by adding the Secure attribute to the cookie when it is set.

Example in PHP:

```
setcookie('cookie_name', 'cookie_value', time() + 3600, '/', '', true, true);
```

Example in Java Servlet:

```
Cookie cookie = new Cookie("cookie_name", "cookie_value");  
cookie.setSecure(true);  
response.addCookie(cookie);
```

- 2. Implement HTTPS:** Use HTTPS instead of HTTP to encrypt the communication between the client and the server. This ensures that all data, including cookies, is transmitted securely.
- 3. Review and update cookie handling:** Review the codebase to identify any instances where cookies are being set without the secure flag. Update the code to include the secure flag for those cookies.

About

The vulnerability "Cookie Without Secure Flag" occurs when a cookie is set without the secure flag. The secure flag is an attribute of a cookie that instructs the browser to only send the cookie over an encrypted (HTTPS) connection. When a cookie is set without the secure flag, it can be accessed by an attacker if the user visits a non-encrypted (HTTP) website. This can lead to the exposure of sensitive information contained in the cookie.

Risks

The risks associated with the vulnerability "Cookie Without Secure Flag" are:

- Information exposure:** If a cookie containing sensitive information is set without the secure flag, an attacker can intercept the cookie by eavesdropping on the network traffic. This can lead to the exposure of user credentials, session tokens, or other sensitive data.
- Session hijacking:** By intercepting a cookie without the secure flag, an attacker can hijack the user's session and impersonate them on the website. This can allow the attacker to perform actions on behalf of the user, potentially leading to unauthorized access or data manipulation.
- Man-in-the-middle attacks:** Without the secure flag, an attacker can perform man-in-the-middle attacks by intercepting the communication between the user and the server. This can enable the attacker to modify the contents of the cookie or inject malicious code into the user's session.

It is important to address this vulnerability to ensure the confidentiality and integrity of user data and to prevent unauthorized access to sensitive information.

Path	Method	Status
/invalidsession	GET	NEW
/sitemap.xml	GET	NEW

Cookie Without SameSite Attribute

Criticality: Low CWE ID: 1275

Description:

Remediation

To remediate the vulnerability of "Cookie Without SameSite Attribute," the SameSite attribute should be added to the cookie. The SameSite attribute is an effective countermeasure to cross-site request forgery (CSRF), cross-site script inclusion (XSSI), and timing attacks. By setting the SameSite attribute to "Strict" or "Lax," the cookie will only be sent in a first-party context, preventing it from being sent in a cross-site request.

To add the SameSite attribute to a cookie, the following steps can be taken:

- 1. Identify the cookies:** Identify the cookies that need to have the SameSite attribute added.
- 2. Update the cookie settings:** Update the cookie settings to include the SameSite attribute with a value of "Strict" or "Lax." The "Strict" value ensures that the cookie is only sent in a first-party context, while the "Lax" value allows the cookie to be sent in a cross-site context if it is triggered by a top-level navigation.

Example configuration in PHP:

```
setcookie('cookie_name', 'cookie_value', ['SameSite' => 'Strict']);
```

Example configuration in JavaScript:

```
document.cookie = 'cookie_name=cookie_value; SameSite=Strict';
```

- 3. Test and verify:** Test the application to ensure that the SameSite attribute has been added to the cookies and that they are only sent in the appropriate context.

About

The vulnerability of "Cookie Without SameSite Attribute" occurs when a cookie is set without the SameSite attribute. This attribute is used to control how cookies are sent in cross-site requests. Without the SameSite attribute, the cookie can be sent as a result of a "cross-site" request, which can lead to security vulnerabilities such as CSRF, XSSI, and timing attacks.

The SameSite attribute can be set to two values:

- Strict:** The cookie will only be sent in a first-party context, meaning it will not be sent in a cross-site request.
 - Lax:** The cookie will be sent in a cross-site context if it is triggered by a top-level navigation. This is the default behavior if the SameSite attribute is not specified.
- By adding the SameSite attribute to cookies, web applications can protect against CSRF attacks, prevent the inclusion of malicious scripts from other domains, and mitigate timing attacks.

Risks

The risks associated with the vulnerability of "Cookie Without SameSite Attribute" include:

- 1. Cross-Site Request Forgery (CSRF):** Without the SameSite attribute, cookies can be sent in cross-site requests, making the application vulnerable to CSRF attacks. Attackers can trick users into performing unintended actions on the application by exploiting the trust between the user's browser and the application.
- 2. Cross-Site Script Inclusion (XSSI):** Cookies without the SameSite attribute can be included in cross-site requests, allowing attackers to include malicious scripts from other domains. This can lead to the execution of unauthorized actions or the theft of sensitive information.
- 3. Timing Attacks:** Timing attacks can be performed by analyzing the timing of requests and responses. Without the SameSite attribute, cookies can be sent in cross-site requests, potentially exposing sensitive information or allowing attackers to gather information about the application's behavior.

By adding the SameSite attribute to cookies, these risks can be mitigated, ensuring that cookies are only sent in a first-party context and protecting the application from CSRF, XSSI, and timing attacks.

Path	Method	Status
/invalidsession	GET	NEW
/sitemap.xml	GET	NEW

Cookie No HttpOnly Flag

Criticality: Low CWE ID: 1004

Description:

Remediation

To remediate the "Cookie No HttpOnly Flag" vulnerability, the following steps can be taken:

- 1. Enable the HttpOnly flag:** Set the HttpOnly flag on all cookies to prevent them from being accessed by JavaScript. This can be done by adding the HttpOnly attribute to the Set-Cookie header when the cookie is being set. For example, in a web application using PHP, the `setcookie` function can be used with the `httponly` parameter set to `true`:

```
setcookie('cookie_name', 'cookie_value', time() + 3600, '/', '',  
false, true);
```

This will ensure that the cookie is only accessible over HTTP and cannot be accessed by JavaScript.

- 2. Review and update cookie handling:** Review the codebase to ensure that cookies are being handled securely. Make sure that sensitive information is not being stored in cookies and that cookies are properly validated and sanitized before being used.
- 3. Implement secure session management:** If the vulnerable cookie is a session cookie, consider implementing additional security measures such as session expiration, session regeneration after login, and using secure session storage mechanisms.

About

The "Cookie No HttpOnly Flag" vulnerability occurs when a cookie is set without the HttpOnly flag. The HttpOnly flag is a security feature that prevents cookies from being accessed by JavaScript. By default, cookies can be accessed by both the server and client-side scripts, including malicious scripts. If a malicious script is able to run on the same page as the vulnerable cookie, it can access and potentially steal the cookie's value. This can lead to session hijacking, where an attacker gains unauthorized access to a user's session.

Risks

The risks associated with the "Cookie No HttpOnly Flag" vulnerability include:

- 1. Session hijacking:** If the vulnerable cookie is a session cookie, an attacker can potentially hijack a user's session by stealing the cookie's value. This can allow the attacker to impersonate the user and perform actions on their behalf.

- 2. Information disclosure:** If the cookie contains sensitive information, such as user credentials or personal data, an attacker can access and steal this information. This can lead to identity theft, unauthorized access to accounts, or other privacy breaches.
- 3. Cross-site scripting (XSS) attacks:** The ability to access cookies through JavaScript can be exploited in conjunction with other vulnerabilities, such as cross-site scripting (XSS). An attacker can inject malicious scripts into a vulnerable page, which can then access and steal the cookie's value. This can lead to further attacks, such as session hijacking or data manipulation.

Path	Method	Status
/invalidsession	GET	NEW
/sitemap.xml	GET	NEW

Permissions Policy Header Not Set

Criticality: Low CWE ID: 693

Description:

Remediation

To fix the "Permissions Policy Header Not Set" vulnerability, you need to add the Permissions-Policy header to your web server configuration. This header allows you to specify which browser features can be used by your web resources. By setting appropriate permissions, you can restrict unauthorized access or usage of sensitive features such as camera, microphone, location, and full screen.

To add the Permissions-Policy header, follow these steps:

1. Apache: Add the following line to your .htaccess file or Apache configuration file:

```
Header set Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()"
```

2. Nginx: Add the following line to your Nginx configuration file within the http block:

```
add_header Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()";
```

3. IIS: Open the IIS Manager, select your website, and go to the "HTTP Response Headers" feature. Add a new header with the name "Permissions-Policy" and the value "camera=(), microphone=(), geolocation=(), fullscreen=()".

Make sure to adjust the permissions according to your specific requirements. For example, if your website requires camera access, you can remove the camera=() part from the header value.

About

The Permissions Policy Header is an added layer of security that helps restrict unauthorized access or usage of browser/client features by web resources. It ensures user privacy by limiting or specifying which features of the browser can be used by web resources. By setting the Permissions-Policy header, website owners can control access to features such as camera, microphone, location, and full screen.

Risks

If the Permissions Policy Header is not set, it can lead to potential security risks. Without proper restrictions, malicious actors may exploit browser features to gain unauthorized access to sensitive user information or perform unauthorized actions. For example, an attacker could abuse

camera or microphone access to capture audio/video without user consent, or misuse geolocation data for tracking purposes. By not setting the Permissions Policy Header, you leave your website and users vulnerable to these risks.

Path	Method	Status
[ROOT]	GET	NEW
/index	GET	NEW
/invalidsession	GET	NEW
/jakarta.faces.resource/css/bootstrap.min.css	GET	NEW
/jakarta.faces.resource/css/oneui.css	GET	NEW
/jakarta.faces.resource/theme.css?ln=primefaces-saga&v=13.0.6	GET	NEW
/robots.txt	GET	NEW
/sitemap.xml	GET	NEW